

DOCUMENTAÇÃO TÉCNICA DE PROJETO: API SOB ALTA DEMANDA

1. INTRODUÇÃO E ESCOPO DO PROBLEMA

1.1 Contextualização do Desafio

O presente projeto aborda o desenvolvimento de uma API (Application Programming Interface) projetada para operar sob cenários de alta concorrência e demanda simultânea. O modelo de negócios baseia-se em um sistema de inventário limitado, simulando um cenário onde estão disponíveis exatamente 100 unidades globais de um determinado produto, com uma regra de negócio restritiva: cada usuário autenticado pode adquirir, no máximo, 2 unidades.

1.2 O Problema da Condição de Corrida (*Race Condition*)

Em sistemas distribuídos ou concorrentes mal estruturados, a latência entre as operações de leitura e escrita no banco de dados pode gerar inconsistências graves de inventário. Se duas requisições simultâneas consultarem a disponibilidade de estoque antes de efetivarem a baixa, ambas receberão uma confirmação positiva e prosseguirão para a venda. Como resultado, o sistema processará mais vendas do que o estoque físico real suporta (conhecido como *overselling* ou sobre demanda ilegal).

O diagrama abaixo ilustra o comportamento anômalo da falha quando não mitigada:

Plaintext

```
+-----+  
----+  
|                                     EXEMPLO DE FALHA DE CONCORRÊNCIA  
|  
+-----+  
----+  
|  
|  
| Requisição A (Usuário 1) -----> [Verifica: Resta 1 vaga]  
|  
| Requisição B (Usuário 2) -----> [Verifica: Resta 1 vaga]  
|  
|  
| Requisição A (Usuário 1) -----> [Efetiva Compra: Estoque =  
0] |  
| Requisição B (Usuário 2) -----> [Efetiva Compra: Estoque = -  
1] |
```

```
|
|
| [ERRO DE CONSISTÊNCIA]: 2 unidades vendidas com apenas 1 em
estoque! |
+-----+
-----+
```

2. ARQUITETURA DA SOLUÇÃO E ENGENHARIA DE DADOS

2.1 Concorrência Baseada em Operações Atômicas

Para mitigar o problema da Condição de Corrida sem a necessidade de implementar travas (*locks*) pessimistas globais na aplicação — o que degradaria drasticamente a performance —, adotou-se o conceito de **Operações Atômicas** diretamente na camada de persistência. A estratégia consiste em unificar as etapas de validação e atualização em uma única instrução indivisível de banco de dados.

Reserva no Pool Global (Limite de 100 Vagas):

SQL

```
UPDATE pool_global
SET vagas_restantes = vagas_restantes - 1
WHERE id = 1 AND vagas_restantes > 0;
```

- **Mecanismo de Controle:** O predicado `WHERE vagas_restantes > 0` atua como uma barreira lógica de checagem em tempo de execução. Se o valor for menor ou igual a zero, a linha não sofre alteração (`changes = 0`), impedindo a sobre venda.

Controle da Cota Individual (Máximo de 2 por Usuário):

SQL

```
UPDATE cota_usuario
SET vagas_ativas = vagas_ativas + 1
WHERE usuario_id = ? AND vagas_ativas < 2;
```

- **Mecanismo de Controle:** Garante que a transação falhará se o usuário tentar alocar uma terceira vaga concorrente, rejeitando a operação na própria instrução de escrita.

2.2 Transacionalidade (Propriedades ACID)

As atualizações de estoque global e cota pessoal são encapsuladas dentro de uma transação de banco de dados para garantir a **Atomicidade** e a **Consistência** do sistema:

- **BEGIN TRANSACTION:** Abre o escopo de isolamento.
- **COMMIT:** Consolida as alterações de forma permanente apenas se ambas as condições (`pool_global` e `cota_usuario`) forem atendidas com sucesso.
- **ROLLBACK:** Reverte integralmente o estado anterior caso qualquer um dos passos falhe ou retorne `changes = 0`, impedindo estados parciais ou dados corrompidos.

3. ESPECIFICAÇÃO DA INFRAESTRUTURA E MODELAGEM DE DADOS

3.1 Stack Tecnológica

A tabela a seguir descreve os componentes do ecossistema e suas respectivas justificativas de adoção no ambiente de desenvolvimento:

Componente	Tecnologia	Justificativa Técnica
Ambiente de Execução	Node.js + Express	Arquitetura assíncrona orientada a eventos, ideal para alta taxa de I/O.
Camada de Persistência	SQLite	Banco de dados relacional leve, embarcado em memória/arquivo, com suporte nativo a transações ACID.
Segurança	JWT (JSON Web Token)	Autenticação baseada em <i>tokens</i> criptográficos <i>stateless</i> , reduzindo consultas ao banco para validação de sessões.
Orquestração	Docker	Padronização e isolamento do ambiente produtivo, eliminando variações de infraestrutura local (" <i>works on my machine</i> ").

3.2 Modelo Entidade-Relacionamento (DDL)

SQL

```
-- Controle do estoque global
CREATE TABLE pool_global (
  id INTEGER PRIMARY KEY,
```

```

        vagas_restantes INTEGER CHECK (vagas_restantes >= 0)
    );

-- Controle da cota de compras por usuário
CREATE TABLE cota_usuario (
    usuario_id INTEGER PRIMARY KEY,
    vagas_ativas INTEGER CHECK (vagas_ativas >= 0 AND vagas_ativas
<= 2),
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);

-- Registro transacional das compras efetuadas
CREATE TABLE entidades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario_id INTEGER,
    produto_id INTEGER,
    status TEXT CHECK (status IN ('RASCUNHO', 'PROCESSANDO',
'CONCLUÍDO')),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);

-- Gerenciamento de credenciais de usuários
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL -- Armazena estritamente o hash
criptográfico da senha
);

```

4. MAPEAMENTO DE FLUXO E CICLO DE VIDA DA REQUISIÇÃO

4.1 Fluxograma Algorítmico do Endpoint /submeter

1.Entrada da Requisição:POST /entidades/{id}/submeter.

A requisição HTTP é interceptada pelo servidor. O cabeçalho é validado em busca do Token Bearer (JWT).

2.Início do Isolamento Escalar:BEGIN TRANSACTION.

O banco de dados inicia uma transação exclusiva para isolar os comandos subsequentes de outras requisições concorrentes.

3.Tentativa de Alocação de Recurso Global:UPDATE pool_global.

Executa a dedução atômica no estoque central. O sistema avalia o número de linhas modificadas (changes). Se `changes == 0`, a operação sofre **ROLLBACK** instantâneo e retorna o código HTTP 409 Conflict (P00L_CHEIO).

4.Tentativa de Alocação de Cota Pessoal:UPDATE cota_usuario.

Caso haja estoque global, incrementa a cota do usuário de forma atômica. Se o usuário violar o teto regulatório de 2 unidades (`changes == 0`), a transação sofre **ROLLBACK** completo e retorna o código HTTP 409 Conflict (COTA_PESSOAL).

5.Finalização e Consistência de Dados:COMMIT.

As alterações são validadas, a tabela entidades muda seu estado de RASCUNHO para PROCESSANDO e o banco consolida os dados. O servidor responde com o código HTTP 200 OK.

4.2 Máquina de Estados da Compra (Ciclo de Vida)

O ciclo de vida de uma transação de compra segue estritamente a transição linear descrita a seguir:

Plaintext

```
[RASCUNHO] -----> [PROCESSANDO] -----> [CONCLUÍDO]
(Pedido Criado)      (Vaga Alocada no Pool)      (Transação
Finalizada)
```

5. DOCUMENTAÇÃO DA INTERFACE DA API (ENDPOINTS)

5.1 Endpoints de Autenticação (Públicos)

- POST /auth/registrar - Cadastro de novos usuários no sistema.
- POST /auth/login - Validação de credenciais e emissão do token JWT.

5.2 Endpoints de Negócio (Requer Token Bearer JWT)

- POST /entidades - Cria uma nova intenção de compra sob o estado inicial de RASCUNHO.
- POST /entidades/{id}/submeter - **Ponto de Alta Concorrência**. Aloca de forma atômica uma vaga no pool e na cota.
- POST /entidades/{id}/finalizar - Conclui o processamento, consolidando a transição de estado da entidade para CONCLUÍDO.

5.3 Endpoints Administrativos e de Auditoria (Requer Token JWT)

- GET /admin/pool - Retorna o estado atual de vagas disponíveis no pool global.

- GET /admin/minha-cota - Retorna as informações transacionais e volumétricas do usuário requisitante.

6. PADRONIZAÇÃO DE RESPOSTAS HTTP E TRATAMENTO DE ERROS

6.1 Estruturas de Sucesso (JSON)

JSON

// HTTP 201 Created - POST /entidades

```
{  
  "id": 42,  
  "status": "RASCUNHO"  
}
```

// HTTP 200 OK - POST /entidades/42/submeter

```
{  
  "id": 42,  
  "status": "PROCESSANDO"  
}
```

// HTTP 200 OK - POST /entidades/42/finalizar

```
{  
  "id": 42,  
  "status": "CONCLUÍDO"  
}
```

6.2 Estruturas de Falha e Regras de Negócio Violadas

JSON

// HTTP 409 Conflict - Tentativa de alocação além do limite global de 100 vagas

```
{  
  "error": "POOL_CHEIO"  
}
```

// HTTP 409 Conflict - Tentativa de alocação que ultrapassa o teto de 2 unidades por usuário

```
{  
  "error": "COTA_PESSOAL"  
}
```

// HTTP 409 Conflict - Conflito cadastral de chaves no banco de

```
dados
{
  "error": "USUARIO_JA_EXISTE"
}

// HTTP 401 Unauthorized - Credenciais incompatíveis com a base de
dados
{
  "error": "CREDENCIAIS_INVALIDAS"
}

// HTTP 401 Unauthorized - Assinatura corrompida ou token JWT
expirado
{
  "error": "TOKEN_INVALIDO"
}
```

7. VALIDAÇÃO EXPERIMENTAL E CADERNO DE TESTES

A API foi submetida a baterias de testes automatizados para atestar o comportamento lógico e a integridade sob estresse:

Caso de Teste 1: Validação de Cota Limite Individual (Usuário Único)

- **Procedimento:** Um usuário realizou três requisições sequenciais de submissão de compra.
- **Resultados Obtidos:** As requisições 1 e 2 retornaram status 200 OK. A requisição 3 retornou status 409 Conflict (COTA_PESSOAL), comprovando a eficácia da restrição programática.

Caso de Teste 2: Esgotamento Volumétrico do Pool Global

- **Procedimento:** Simulação de vendas contínuas até atingir o limite programado de 100 registros.
- **Resultados Obtidos:** A centésima primeira (101ª) tentativa foi sumariamente rejeitada com o status 409 Conflict (POOL_CHEIO). Após a execução de uma rota de liberação (/finalizar), o pool reajustou a contagem e aceitou uma nova inserção com sucesso.

Caso de Teste 3: Simulação de Alta Concorrência em Larga Escala

- **Procedimento:** Disparo simultâneo de **110 requisições assíncronas concorrentes** (via testes de estresse) disputando as últimas vagas livres do sistema.
- **Métricas de Controle:**
 - **Sucessos Computados:** Exatamente 100 requisições processadas com status 200 OK.
 - **Rejeições Computadas:** Exatamente 10 requisições tratadas com status 409 Conflict.
 - **Integridade Física do Banco:** Nenhuma vaga duplicada, nenhuma venda fantasma registrada e nenhum travamento de conexões (*deadlock*).

8. ANÁLISE DE ATRIBUTOS DE QUALIDADE DA SOLUÇÃO (MÉTRICAS)

- **Corretude (*Correctness*):** Garantida pelo acoplamento das validações de restrição (`vagas_restantes > 0` e `vagas_ativas < 2`) no escopo imperativo das cláusulas `WHERE`. O sistema se autoprotege contra falhas lógicas no nível da persistência.
- **Justiça (*Fairness*):** As requisições concorrentes são processadas de maneira sequencial e justa pela fila interna do mecanismo do banco de dados relacional. O primeiro pacote de dados a atingir o bloqueio lógico da linha garante a reserva do produto (*First-Come, First-Served*).
- **Eficiência e Desempenho:** A substituição de travas completas de tabela (*Table Locks*) por atualizações diretas de registros específicos garante baixa latência no processamento. As transações curtas minimizam o tempo de retenção de memória e otimizam a vazão global do servidor.
- **Clareza e Padronização:** O projeto adota uma arquitetura limpa com mensagens normatizadas de erro e semântica REST rigorosa através do uso correto dos códigos de status do protocolo HTTP.

9. MANUAL DE IMPLANTAÇÃO, EXECUÇÃO E DEPLOY

1. PRÉ-REQUISITOS DE INFRAESTRUTURA

Para a correta inicialização e execução do ecossistema da API, o ambiente operacional deve dispor das ferramentas especificadas na tabela abaixo:

Dependência	Versão Mínima	Finalidade Sistêmica	Link para Download Oficial
-------------	---------------	----------------------	----------------------------

Docker Desktop	4.0+	Orquestração de containers e isolamento do ambiente.	docker.com/products/docker-desktop
Git Client	2.0+	Controle de versão e clonagem do código-fonte.	git-scm.com/downloads
Interface CLI / Terminal	Native	Terminal para execução de comandos (Bash, PowerShell ou CMD).	Nativo no Sistema Operacional

Nota de Configuração: Certifique-se de que o daemon do Docker esteja completamente inicializado e ativo em segundo plano antes de disparar os comandos de compilação na interface de linha de comando (CLI).

2. PROCEDIMENTOS DE IMPLANTAÇÃO PASSO A PASSO

Passo 1: Obtenção do Código-Fonte

Execute o comando abaixo no terminal de sua preferência para clonar os arquivos do projeto a partir do repositório remoto:

```
Bash
# Clonar o repositório hospedado no GitHub
git clone https://github.com/seu-usuario/alta-demanda-api.git

# Acessar o diretório raiz do projeto implantado
cd alta-demanda-api
```

Nota: Em ambientes restritos onde o cliente Git não esteja disponível, o código-fonte poderá ser obtido via download do arquivo compactado (.ZIP) e extraído manualmente no diretório de trabalho.

Passo 2: Validação da Árvore de Diretórios

Antes de iniciar a compilação, certifique-se de que a estrutura básica do projeto contém os seguintes arquivos essenciais:

Plaintext

alta-demanda-api/	
├─ Dockerfile	- Manifesto de configuração do container
base	
├─ docker-compose.yml	- Arquivo de orquestração multifuncional
dos serviços	
├─ .dockerignore	- Diretiva de exclusão de arquivos no
escopo do build	
├─ package.json	- Manifesto de gerenciamento de
dependências Node.js	
├─ server.js	- Ponto de entrada (script principal) da
API REST	
├─ database.js	- Camada de persistência e conexões com
o SQLite	
└─ auth.js	- Regras de validação e emissão de
tokens JWT	

Para validar a listagem de arquivos localmente, execute:

Bash

```
# Em ambientes baseados em Windows (PowerShell):
dir
```

```
# Em ambientes baseados em Unix/Linux/macOS:
ls -la
```

Passo 3: Orquestração e Inicialização da API via Docker

Inicie o processo de construção das camadas de imagem e execução do container em modo isolado através do utilitário Docker Compose:

Bash

```
# Compilar as imagens e inicializar os serviços em segundo plano
(Detached Mode)
docker compose up --build -d
```

- `--build`: Garante a invalidação do cache e força a reconstrução estrutural da imagem a cada modificação do código-fonte.
- `-d`: Libera o terminal de origem, executando a stack de serviços de forma assíncrona.

Para realizar a inspeção das saídas de texto (logs) do servidor em tempo real, utilize:

Bash

```
docker compose logs -f
```

Para auditar a saúde da instância e o mapeamento correto de portas lógicas, execute:

Bash

```
docker ps
```

Saída terminal esperada:

Plaintext

CONTAINER ID	IMAGE	STATUS	PORTS
abc123def456	alta-demanda-api-api	Up 2 minutes	0.0.0.0:8080->8080/tcp

Passo 4: Ensaio de Conectividade Inicial (Smoke Test)

Bash

```
# Disparo de requisição para certificar que o servidor HTTP está em estado de escuta
```

```
curl http://localhost:8080/auth/login -X POST
```

Nota Diagnóstica: O retorno de um erro do tipo `connection refused` indica que o container não foi inicializado corretamente ou que a porta lógica 8080 do sistema hospedeiro está sofrendo concorrência com outro serviço ativo.

Passo 5: Provisionamento e Registro de Usuários

Bash

```
curl -X POST http://localhost:8080/auth/registrar \
  -H "Content-Type: application/json" \
  -d '{"username":"joao","password":"123"}'
```

Retorno esperado do servidor (HTTP 201 Created):

JSON

```
{
  "id": 1,
  "username": "joao"
}
```

Passo 6: Autenticação Baseada em Credenciais e Emissão de Token JWT

Bash

```
curl -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"joao","password":"123"}'
```

Retorno esperado do servidor (HTTP 200 OK):

JSON

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Atenção: O valor da propriedade token deve ser preservado para inclusão no cabeçalho de requisições que exijam privilégios de acesso.

Automatização do armazenamento do token em ambiente Windows PowerShell:

PowerShell

```
$login = curl -X POST http://localhost:8080/auth/login -H "Content-Type: application/json" -d '{"username":"joao","password":"123"}'
$token = ($login.Content | ConvertFrom-Json).token
```

Passo 7: Instanciação de Requisições sob o Estado de RASCUNHO

Cada intenção de compra inicia obrigatoriamente sob o estado de RASCUNHO. Execute os comandos abaixo para instanciar três registros de teste:

Bash

Instanciação do Registro 01

```
curl -X POST http://localhost:8080/entidades \
  -H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>" \
  -H "Content-Type: application/json" \
  -d '{"produtoId":12}'
```

Instanciação do Registro 02

```
curl -X POST http://localhost:8080/entidades \
  -H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>" \
  -H "Content-Type: application/json" \
  -d '{"produtoId":12}'
```

Instanciação do Registro 03

```
curl -X POST http://localhost:8080/entidades \  
  -H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>" \  
  -H "Content-Type: application/json" \  
  -d '{"produtoId":12}'
```

Retorno estrutural para cada inserção bem-sucedida:

```
JSON  
{  
  "id": 1,  
  "status": "RASCUNHO"  
}
```

Passo 8: Validação Empírica das Restrições de Cota Pessoal (Teto por Usuário)

Submeta sequencialmente as requisições geradas para comprovar que o sistema bloqueia tentativas que excedam o limite máximo de duas alocações por usuário:

Bash

Submissão da Entidade ID 1 (Sucesso Operacional)

```
curl -X POST http://localhost:8080/entidades/1/submeter \  
  -H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>"
```

Submissão da Entidade ID 2 (Sucesso Operacional)

```
curl -X POST http://localhost:8080/entidades/2/submeter \  
  -H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>"
```

Submissão da Entidade ID 3 (Rejeição por Violação de Cota)

```
curl -X POST http://localhost:8080/entidades/3/submeter \  
  -H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>"
```

Resultados esperados das requisições:

- Entidade 1: HTTP 200 OK -> {"id":1,"status":"PROCESSANDO"}
- Entidade 2: HTTP 200 OK -> {"id":2,"status":"PROCESSANDO"}
- Entidade 3: HTTP 409 Conflict -> {"error":"COTA_PESSOAL"}

Passo 9: Auditoria do Inventário Global e Estado das Cotas

Bash

Consulta ao quantitativo remanescente do estoque central (Pool Global)

```
curl -X GET http://localhost:8080/admin/pool \  

```

```
-H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>"
```

Consulta ao volume de alocações concorrentes vinculadas ao usuário autenticado

```
curl -X GET http://localhost:8080/admin/minha-cota \  
-H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>"
```

Respostas métricas esperadas:

JSON

```
{ "vagas_restantes": 98 }  
{ "vagas_ativas": 2 }
```

Passo 10: Conclusão de Ciclo de Vida e Desalocação Dinâmica de Recursos

Ao finalizar o processamento de uma entidade, a vaga por ela ocupada deve retornar ao pool central:

Bash

```
# Alterar o estado da Entidade ID 1 para finalizado  
curl -X POST http://localhost:8080/entidades/1/finalizar \  
-H "Authorization: Bearer <INSERIR_TOKEN_JWT_AQUI>"
```

Resposta esperada:

JSON

```
{  
  "id": 1,  
  "status": "CONCLUÍDO"  
}
```

Efeito colateral observado: Ao efetuar uma nova consulta ao endpoint `/admin/pool`, o valor numérico da propriedade `vagas_restantes` será reajustado incrementalmente para 99, comprovando a consistência transacional do ecossistema de dados.

Passo 11: Automação de Carga para Testes de Estresse Sob Concorrência Limite

O script abaixo orquestra múltiplos loops de requisições síncronas/assíncronas para testar o comportamento do pool sob concorrência extrema (110 usuários disputando as 100 vagas configuradas de estoque).

Script para Ambientes Unix / Linux / macOS (Bash Shell):

Bash

```
for i in {1..110}; do
    curl -s -X POST http://localhost:8080/auth/regar -H "Content-Type:
application/json" -d
{"\"username\": \"user$i\", \"password\": \"123\"}" > /dev/null
    TOKEN=$(curl -s -X POST http://localhost:8080/auth/login -H "Content-
Type: application/json" -d
{"\"username\": \"user$i\", \"password\": \"123\"}" | jq -r '.token')
    curl -s -X POST http://localhost:8080/entidades -H "Authorization:
Bearer $TOKEN" -H "Content-Type: application/json" -d
'{"produtoId":12}' > /dev/null
    RESULT=$(curl -s -X POST http://localhost:8080/entidades/1/submeter -H
"Authorization: Bearer $TOKEN")
    echo "Usuário $i: $RESULT"
done
```

Script para Ambientes Windows (PowerShell):

PowerShell

```
for ($i=1; $i -le 110; $i++) {
    $body = @{username="user$i"; password="123"} | ConvertTo-Json
    try {
        Invoke-WebRequest -Uri "http://localhost:8080/auth/regar" -
Method POST -ContentType "application/json" -Body $body -
UseBasicParsing | Out-Null
        $login = Invoke-WebRequest -Uri "http://localhost:8080/auth/login"
-Method POST -ContentType "application/json" -Body $body -
UseBasicParsing
        $t = ($login.Content | ConvertFrom-Json).token
        $ent = Invoke-WebRequest -Uri "http://localhost:8080/entidades" -
Method POST -Headers @{Authorization = "Bearer $t"} -ContentType
"application/json" -Body '{"produtoId":12}' -UseBasicParsing
        $id = ($ent.Content | ConvertFrom-Json).id
        $resp = Invoke-WebRequest -Uri
"http://localhost:8080/entidades/\$id/submeter" -Method POST -Headers
@{Authorization = "Bearer $t"} -UseBasicParsing -ErrorAction Stop
        Write-Host "Usuário $i - SUCESSO (vaga $i)" -ForegroundColor
Green
    }
    catch {
        Write-Host "Usuário $i - ERRO (pool deve estar cheio)" -
ForegroundColor Red
    }
}
```

```
}  
}
```

Resultado do Teste de Carga: As primeiras 100 requisições retornarão confirmação positiva de sucesso de alocação de vaga, enquanto as 10 requisições excedentes receberão a resposta de erro 409 Conflict (POOL_CHEIO).

Passo 12: Encerramento de Instâncias e Purga de Volumes Persistidos

Bash

```
# Encerrar e desativar os serviços em execução nos containers  
docker compose down
```

```
# Encerrar os containers e deletar os volumes associados (Zerar a  
base de dados do SQLite)  
docker compose down -v
```

3. RESOLUÇÃO DE ERROS E INCONSISTÊNCIAS (TROUBLESHOOTING)

Inconsistência Detectada	Causa Provável	Ação Corretiva Sugerida
connection refused	O daemon do Docker encontra-se inativo ou travado.	Certifique-se de que a aplicação Docker Desktop está inicializada adequadamente.
port 8080 already in use	Conflito de alocação de porta de rede no sistema hospedeiro.	Altere o mapeamento de portas externas contido no arquivo docker-compose.yml.
token invalid	Expiração temporal ou corrupção na assinatura do payload criptográfico.	Invoque novamente o endpoint /auth/login para a emissão de um novo token válido.
SQLITE_CANTOPEN	Violação ou ausência de privilégios de escrita no diretório do container.	Execute a derrubada completa e limpa dos volumes via comando docker compose down -v.
curl: command not found	Ausência da ferramenta de transferência de	Instale o pacote curl localmente ou adote o comando nativo

	dados utilitária no sistema.	Invoke-WebRequest do PowerShell.
--	------------------------------	----------------------------------

4. PROTOCOLO DE REVISÃO E HOMOLOGAÇÃO (CHECKLIST)

A tabela abaixo serve como instrumento de validação para verificar a conformidade dos resultados obtidos durante o deploy da solução:

Escopo do Teste	Comando Utilizado	Comportamento Esperado para Aprovação	Status (Homologado)
Status de Execução	docker ps	Exibição da imagem alta-demanda-api-api ativa na porta 8080.	[]
Persistência Cadastral	Rota /auth/registrar	Resposta com o código HTTP 201 contendo o ID incremental gerado.	[]
Emissão de Identidade	Rota /auth/login	Resposta com o código HTTP 200 retornando uma String estruturada JWT.	[]
Instanciação Lógica	Rota /entidades	Resposta com o código HTTP 201 e a propriedade status em RASCUNHO.	[]
Garantia de Cota (Unid 1)	Rota /entidades/1/submeter	Resposta com o código HTTP 200 e alteração para PROCESSANDO.	[]
Garantia de Cota (Unid 2)	Rota /entidades/2/submeter	Resposta com o código HTTP 200 e alteração para PROCESSANDO.	[]

Garantia de Cota (Unid 3)	Rota /entidades/3/submeter	Rejeição automática com o código HTTP 409 e o erro COTA_PESSOAL.	[]
Auditoria do Pool	Rota /admin/pool	Registro decrescente exato indicando vagas_restantes: 98.	[]
Auditoria Individual	Rota /admin/minha-cota	Registro de teto atômico indicando vagas_ativas: 2.	[]
Desalocação Atômica	Rota /entidades/1/finalizar	Resposta com o código HTTP 200 e status atualizado para CONCLUÍDO.	[]
Teste de Saturação	Script de Teste de Carga	Registro de exatos 100 retornos positivos e 10 falhas por esgotamento.	[]